

1. Lists, Links, Images, Tables, Forms.**a. Write a HTML program to explain the working of lists.**

Note: It should have an ordered list, unordered list, nested lists and ordered list in an unordered list and definition lists.

b. Write a HTML program to explain the working of hyperlinks using <a> tag and href, target attributes.

Note: Use text to link → <https://www.aec.edu.in/>

Use image to link → <https://www.aec.edu.in/?p=Gallery>

**c. Create a HTML document that has your image and your friend's image with a specific height and width. Also when clicked on the images it should navigate to their respective profiles.*

d. Write a HTML program to explain the working of tables.

(use tags: <table>, <tr>, <th>, <td> and attributes: border, rowspan, colspan)

e. Write a HTML program to explain the working of forms by designing a Registration form.

(Note: Include text field, password field, number field, date of birth field.)

a. Write a HTML program to explain the working of lists.

Note: It should have an ordered list, unordered list, nested lists and ordered list in an unordered list and definition lists.

Below is a **complete HTML program** that includes:

✓ Ordered List

✓ Unordered List

✓ Nested List

✓ Ordered List inside Unordered List

✓ Definition List

- HTML stands for HyperText Markup Language, and it is used to create web pages.
- It provides the structure of a webpage using tags and elements.
- HTML is not a programming language; it is a markup language.
- All HTML pages start with a <!DOCTYPE html> declaration.
- Web browsers like Chrome and Firefox read and display HTML content.
- HTML does not contain logic (no loops, no conditions).
- It only describes the structure and content of a page.
- Browsers read the HTML file and display it.

Basic HTML Tags

<html>

The root tag of every HTML document.

<head>

Contains metadata (information *about* the webpage).

```
<head>
```

```
  <title>My Web Page</title>
```

```
</head>
```

```
<body>
```

Contains everything visible on the webpage (text, images, lists, tables).

```
</body>
```

```
  <h1>Welcome</h1>
```

```
</body>
```

```
<title>
```

Sets the title of the webpage shown in the browser tab.

```
<meta>
```

Provides information about the webpage like character set, keywords, author.

Meaning of <meta charset="UTF-8">

UTF-8 is a character encoding standard

It tells the browser how to interpret characters (letters, symbols, emojis, languages).

✓ **It supports almost all languages:**

- English
- Telugu
- Hindi
- Tamil
- Kannada
- Arabic
- Chinese
- Emoji characters 😊

Because of this, UTF-8 is the **most widely used encoding** for websites.

Tags with Opening and Closing (Paired Tags)

These have **content between the tags** and always need a closing tag.

Examples:

```
<p>This is a paragraph.</p>    <!-- <p> opening, </p> closing -->
```

```
<h1>Heading</h1>             <!-- <h1> opening, </h1> closing -->
```

```
<div>
```

```
  <p>Nested paragraph.</p>
```

```

</div>

<a href="https://example.com">Link</a>

<ul>

  <li>Item 1</li>

  <li>Item 2</li>

</ul>

```

✓ Rule: Always have a **start tag** <tag> and a **matching end tag** </tag>.

Tags with Only Opening (Self-Closing Tags)

These **don't have content inside** and do not need a closing tag.

Examples:

```

<br>          <!-- Line break -->

<hr>          <!-- Horizontal line -->

 <!-- Image -->

<input type="text" name="username">          <!-- Input field -->

<meta charset="UTF-8">          <!-- Metadata -->

<link rel="stylesheet" href="style.css">      <!-- Stylesheet link -->

```

- You can optionally write them as
, , etc., but in HTML5 it's **not required**.

a. Write a HTML program to explain the working of lists.

Note: It should have an ordered list, unordered list, nested lists and ordered list in an unordered list and definition lists.

- Ordered List () displays items in a numbered sequence.
- Unordered List () shows items using bullet points.
- Nested Lists allow one list to be placed inside another for hierarchical structure.
- Ordered List inside an Unordered List is used to show categories with numbered sub-items.
- Definition List (<dl>) presents terms and their meanings using <dt> and <dd> tags.

```

<!DOCTYPE html>

<html>

<head>

  <title>Working of Lists - Aditya University Example</title>

</head>

<body>

  <h1>Aditya University - Departments & Courses</h1>

```

```

<!-- Ordered List -->

<h2>1. Main Schools in Aditya University (Ordered List)</h2>

<ol>

  <li>School of Engineering</li>

  <li>School of Pharmacy</li>

  <li>School of Business</li>

</ol>

<!-- Unordered List -->

<h2>2. Schools (Unordered List)</h2>

<ul>

  <li>School of Engineering</li>

  <li>School of Pharmacy</li>

  <li>School of Business</li>

</ul>

<!-- Nested List -->

<h2>3. School of Engineering with Branches (Nested List)</h2>

<ul>

  <li>School of Engineering
    <ul>
      <li>AI & ML</li>
      <li>Agricultural Engineering</li>
      <li>Civil Engineering</li>
      <li>Computer Science Engineering (CSE)</li>
      <li>Electronics & Communication Engineering (ECE)</li>
      <li>Electrical & Electronics Engineering (EEE)</li>
      <li>Mechanical Engineering (MECH)</li>
      <li>Mining Engineering</li>
      <li>Petroleum Engineering</li>
    </ul>
  </li>

</ul>

```



ADITYA
UNIVERSITY

```

<!-- Ordered List inside Unordered List -->

<h2>4. AI & ML Branch with Sections (Ordered List inside Unordered List)</h2>

<ul>

  <li>AI & ML Branchs

    <ol>

      <li>Section 1</li>

      <li>Section 2</li>

      <li>Section 3</li>

      <li>Section 4</li>

      <li>Section 5</li>

      <li>Section 6</li>

      <li>Section 7</li>

      <li>Section 8</li>

    </ol>

  </li>

</ul>

<!-- Definition List -->

<h2>5. Definition List of Schools</h2>

<dl>

  <dt>School of Engineering</dt>

  <dd>Offers various engineering programs such as CSE, ECE, EEE, Mech, Civil, AI&ML, etc.</dd>

  <dt>School of Pharmacy</dt>

  <dd>Provides courses related to pharmaceutical sciences.</dd>

  <dt>School of Business</dt>

  <dd>Offers BBA, MBA, and other business-related programs.</dd>

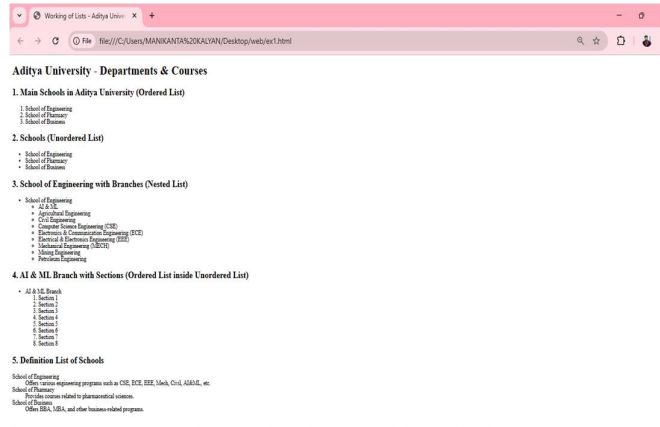
</dl>

</body>

</html>

```

Output:



b. Write a HTML program to explain the working of hyperlinks using <a> tag and href, target attributes.

Note: Use text to link → <https://www.aec.edu.in/>

Use image to link → <https://www.aec.edu.in/?p=Gallery>

<a> Tag (Anchor Tag)

- The <a> tag is used to create **hyperlinks** in HTML.
- Anything inside the <a> tag (text, image, or other content) becomes clickable.
- **Syntax:**

`<a href="URL">Clickable Text or Image</a>`

href Attribute (Hypertext Reference)

- The href attribute specifies the **URL** of the page or resource you want to link to.
- Example:
`<a href="https://www.aec.edu.in/">Visit AEC</a>`
- Without href, the <a> tag won't link anywhere.

target Attribute

- The target attribute defines **where to open the linked page**.
- Common values:
 - `_blank` → Opens the link in a **new tab or window**.
 - `_self` → Opens the link in the **same tab** (default).
- Example:

`<a href="https://www.aec.edu.in/" target="_blank">AEC Website</a>`

- `<a>` → creates a link
- `href` → tells the link **where to go**
- `target` → tells **where to open the link**

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Hyperlinks Example</title>

</head>

<body>

  <h1>Hyperlinks Example</h1>

  <!-- Text link -->

  <p>

    Visit our official website:

    <a href="https://www.aec.edu.in/" target="_blank">AEC Website</a>

  </p>

  <!-- Image link -->

  <p>

    Click the image to see our gallery:

    <a href="https://www.aec.edu.in/?p=Gallery" target="_blank">

    </a>

  </p>

  <p>Explanation:</p>

  <ul>

    <li><strong>href</strong>: Specifies the URL of the page to link to.</li>

    <li><strong>target="_blank"</strong>: Opens the link in a new tab/window.</li>

    <li>You can use text or an image inside the <code>&lt;a></code> tag to create a clickable link.</li>

  </ul>

</body>

</html>

```



***c. Create a HTML document that has your image and your friend's image with a specific height and width. Also when clicked on the images it should navigate to their respective profiles.**

- HTML uses the tag to display images on a webpage.
- The <a> (anchor) tag is used to create hyperlinks that navigate to another page.
- By placing an image inside an <a> tag, we can make the image clickable.
- The height and width attributes help set the image to a specific size.
- This allows us to show photos of people and open their respective profile pages when clicked.

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="utf-8" />
```

```
<meta name="viewport" content="width=device-width,initial-scale=1" />
```

```
<title>Images linking to profiles</title>
```

```
<style>
```

```
/* small style to align images and add spacing */
```

```
.container {
```

```
display: flex;
```

```
gap: 24px;
```

```
align-items: center;
```

```
padding: 24px;
```

```
font-family: Arial, sans-serif;
```

```
}
```

```
.profile {
```



```

text-align: center;

width: 160px; /* container width for each profile card */
}

.profile img {
  display: block;
  border-radius: 8px;
  object-fit: cover; /* keeps image aspect inside given width/height */
  width: 150px; /* required width */
  height: 150px; /* required height */
  cursor: pointer;
  box-shadow: 0 2px 6px rgba(0,0,0,0.12);
}

.caption {
  margin-top: 8px;
  font-size: 14px;
}
</style>
</head>
<body>
<div class="container">
  <!-- YOUR PROFILE -->
  <div class="profile">
    <!-- Replace href with your profile URL and src with your image path/URL -->
    <a href="https://example.com/your-profile" target="_blank" rel="noopener noreferrer" title="Go to my profile">
      
    </a>
    <div class="caption">Your Name</div>
  </div>

```

```

<!-- FRIEND'S PROFILE -->

<div class="profile">

  <!-- Replace href with friend's profile URL and src with friend's image path/URL -->

  <a href="https://example.com/friend-profile" target="_blank" rel="noopener noreferrer" title="Go
to friend's profile">

  </a>

  <div class="caption">Friend's Name</div>

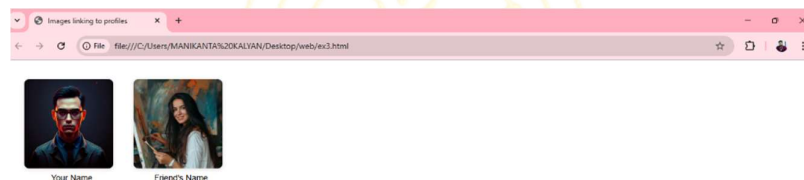
</div>

</div>

</body>

</html>

```



d. Write a HTML program to explain the working of tables.

(use tags: `<table>`, `<tr>`, `<th>`, `<td>` and attributes: `border`, `rowspan`, `colspan`)

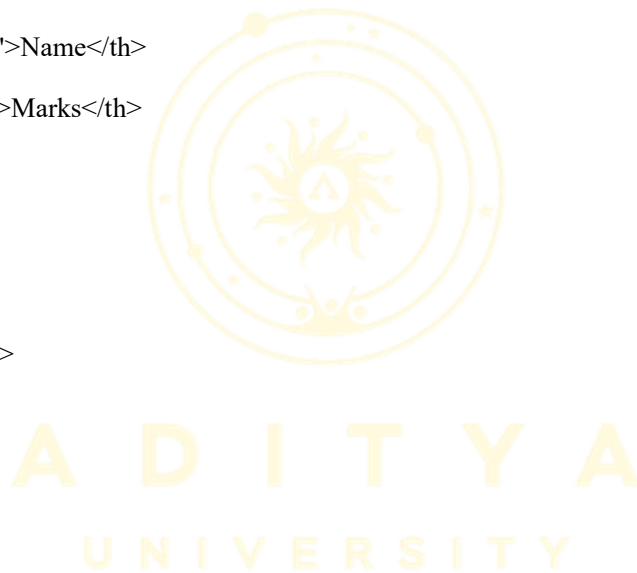
- `<table>` → Used to create a table in HTML.
- `<tr>` (Table Row) → Defines a row inside the table.
- `<th>` (Table Header) → Defines header cells (bold + centered).
- `<td>` (Table Data) → Defines normal data cells.

□ Attributes:

- **border** → Adds borders to the table.
- **rowspan** → A cell spans multiple rows.
- **colspan** → A cell spans multiple columns.

```
<!DOCTYPE html>
```

```
<html>
<head>
  <title>Student Table Example</title>
</head>
<body>
<h2>Student Data Table Example</h2>
<table border="1">
  <!-- Header Row -->
  <tr>
    <th rowspan="2">Student ID</th>
    <th rowspan="2">Name</th>
    <th colspan="2">Marks</th>
  </tr>
  <tr>
    <th>Maths</th>
    <th>Science</th>
  </tr>
  <!-- Row 1 -->
  <tr>
    <td>101</td>
    <td>Ravi</td>
    <td>85</td>
    <td>90</td>
  </tr>
  <!-- Row 2 -->
  <tr>
    <td>102</td>
    <td>Priya</td>
```



```

        <td>92</td>

        <td>88</td>

    </tr>

    <!-- Row 3 -->

    <tr>

        <td>103</td>

        <td>Karthik</td>

        <td>76</td>

        <td>80</td>

    </tr>

</table>

</body>

</html>

```



Student Data Table Example

Student ID	Name	Marks	
		Maths	Science
101	Ravi	85	90
102	Priya	92	88
103	Karthik	76	80

e. Write a HTML program to explain the working of forms by designing a Registration form.

(Note: Include text field, password field, number field, date of birth field.)

1. **HTML forms are used to collect user input**, and a registration form is one of the most common examples used on websites.
2. The registration form allows users to enter personal information using different **input types** such as text, password, number, and date.

3. **Text fields** are used for entering names or usernames, **password fields** hide the characters, **number fields** accept only numeric input, and the **DOB field** helps users select their birth date.
4. By using tags like `<form>`, `<label>`, `<input>`, and attributes like `type=""`, developers can create interactive and user-friendly input sections.
5. This program demonstrates how different input types work together in a simple registration page, showing the basic structure and functioning of HTML forms.

```
<!doctype html>

<html>

<body style="text-align:center">

<form>

  Name: <input type="text"><br><br>
  Age: <input type="number"><br><br>
</form>

<form>

  Password: <input type="password"><br><br>
  DOB: <input type="date"><br><br>
</form>

<form>

  Email: <input type="text"><br><br>
  <button type="submit">Submit</button>
</form>

</body>

</html>
```



The screenshot shows a web browser window with a single tab titled 'ex5.html'. The address bar displays the file path 'file:///C:/Users/MANIKANTA%20KALYAN/Desktop/web/ex5.html'. The webpage content consists of a form with the following elements:

- A label 'Name:' followed by a text input field.
- A label 'Age:' followed by a text input field.
- A label 'Password:' followed by a text input field.
- A label 'DOB:' followed by a date picker control showing 'dd-mm-yyyy'.
- A label 'Email:' followed by a text input field.
- A 'Submit' button at the bottom of the form.

2. HTML 5 and Cascading Style Sheets, Types of CSS & CSS Box Model

a. Write a HTML program, that makes use of <article>, <aside>, <figure>, <figcaption>, <footer>, <header>, <main>, <nav>, <section>, <div>, tags.

- **Semantic Structure:** Tags like `<header>`, `<main>`, `<section>`, `<article>`, and `<footer>` provide clear, meaningful structure to the webpage.
- **Navigation Support:** The `<nav>` tag is used to create a navigation bar for easy movement between pages.
- **Media Representation:** `<figure>` and `<figcaption>` together display an image with a caption, improving accessibility.
- **Supplementary Content:** The `<aside>` tag is used to show related or side information alongside the main content.
- **Generic Containers:** `<div>` is used as a block-level container and `<span>` as an inline text highlighter.

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<title>HTML5 Semantic Tags Example</title>

<style>

    body {

        font-family: Arial, sans-serif;

    }

    nav a {

        margin-right: 15px;

        text-decoration: none;
```

```

        color: darkblue;
    }
    aside {
        background-color: #f2f2f2;
        padding: 10px;
        width: 30%;
    }
    .highlight {
        color: green;
        font-weight: bold;
    }
    figure {
        margin: 10px 0;
    }
</style>
</head>
<body>
<header>
    <h1>Aditya University – Web Technology Demo</h1>
    <!-- Navigation Links -->
    <nav>
        <a href="https://adityauniversity.in" target="_blank">Home</a>
        <a href="https://adityauniversity.in/about-us/" target="_blank">About Us</a>
        <a href="https://adityauniversity.in/academics/" target="_blank">Academics</a>
        <a href="https://adityauniversity.in/admissions/" target="_blank">Admissions</a>
        <a href="https://adityauniversity.in/contact-us/" target="_blank">Contact</a>
    </nav>
</header>

```



A D I T Y A
U N I V E R S I T Y

```

<main>

  <section>

    <h2>University Overview</h2>

    <article>

      <h3>About Aditya University</h3>

      <p>

        <span class="highlight">Aditya University</span> is a premier institution
        offering quality education and research opportunities.

      </p>

      <figure>

        <figcaption>Figure: Aditya University Logo</figcaption>

      </figure>

    </article>

  </section>

  <!-- Aside Section with Inner Links -->

  <aside>

    <h3>Quick Links</h3>

    <ul>

      <li>

        <a href="https://adityauniversity.in/engineering/" target="_blank">

          Engineering Programs

        </a>

      </li>

      <li>

        <a href="https://adityauniversity.in/placements/" target="_blank">

          Placements

        </a>

      </li>
    </ul>
  </aside>

```



```

</li>

<li>

  <a href="https://adityauniversity.in/examinations/" target="_blank">

    Examinations

  </a>

</li>

<li>

  <a href="https://adityauniversity.in/research/" target="_blank">

    Research

  </a>

</li>
</ul>
</aside>
<div>

  <p>

    This <span class="highlight">div</span> is used as a general container

    for additional information.

  </p>
</div>

</main>
<footer>

  <p>

    © 2025 Aditya University |

    <a href="https://adityauniversity.in/privacy-policy/" target="_blank">

      Privacy Policy

    </a>

  </p>

```

</footer>

</body>

</html>

Aditya University – Web Technology Demo

[Home](#) [About Us](#) [Academics](#) [Admissions](#) [Contact](#)

University Overview

About Aditya University

Aditya University is a premier institution offering quality education and research opportunities.

 Aditya University Logo
Figure: Aditya University Logo

Quick Links

- [Engineering Programs](#)
- [Placements](#)
- [Examinations](#)
- [Research](#)

This **div** is used as a general container for additional information.

© 2025 Aditya University | [Privacy Policy](#)

b. Write a HTML program, to embed audio and video into HTML web page.

- Place index.html, sample-audio.mp3, and sample-video.mp4 in the same folder
- Double-click index.html
- The page opens in a browser
- Click Play on audio and video
 - ✚ <audio> tag is used to embed sound
 - ✚ <video> tag is used to embed video
 - ✚ controls attribute shows play, pause, volume
 - ✚ <source> tag specifies the media file

<!DOCTYPE html>

<html>

<head>

<title>Embedding Audio and Video</title>

</head>

<body>

```
<h1>Embedding Audio and Video in HTML</h1>
```

```
<h2>Audio Example</h2>
```

```
<audio controls>
```

```
  <source src="sample-audio.mp3" type="audio/mpeg">
```

Your browser does not support the audio element.

```
</audio>
```

```
<h2>Video Example</h2>
```

```
<video width="400" height="300" controls>
```

```
  <source src="sample-video.mp4" type="video/mp4">
```

Your browser does not support the video element.

```
</video>
```

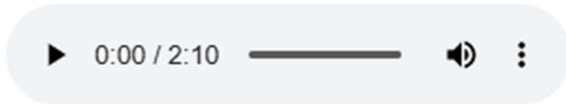
```
</body>
```

```
</html>
```

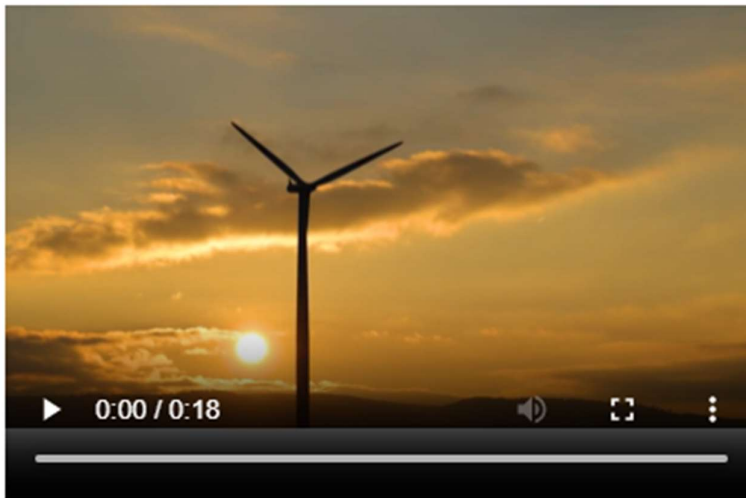


Embedding Audio and Video in HTML

Audio Example



Video Example



c. Write a program to demonstrate the various ways you can reference a color in CSS.

CSS (Cascading Style Sheets) allows developers to apply colors to web page elements such as text, background, borders, and more.

There are **multiple ways to define colors in CSS**, which provide flexibility and better control over design and appearance.

Understanding different color representation methods helps students create visually appealing and professional web pages.

The commonly used ways are:

1. **Color Names**
2. **Hexadecimal Values**
3. **RGB Values**
4. **RGBA Values**
5. **HSL Values** HSL stands for
Hue, Saturation, Lightness
Hue: The color type (0°–360° on the color wheel)

Saturation: Intensity of the color (0%–100%)

Lightness: Brightness of the color (0%–100%)

6. HSLA Values

Hue, Saturation, Lightness, Alpha

Alpha: Transparency level (0.0 to 1.0)

1. 0 = fully transparent

2. 1 = fully opaque

```
<!DOCTYPE html>

<html>

<head>

  <title>CSS Color Representation - Aditya University</title>

  <style>

    body {

      font-family: Arial, sans-serif;

    }

    /* Color Name */

    .color-name {

      color: blue;

    }

    /* Hexadecimal Color */

    .hex-color {

      color: #8B0000;

    }

    /* RGB Color */

    .rgb-color {

      color: rgb(0, 128, 0);

    }

    /* RGBA Color */

    .rgba-color {

      color: rgba(255, 69, 0, 0.7);

    }

    /* HSL Color */
```



ADITYA
UNIVERSITY

```
.hsl-color {  
  color: hsl(240, 100%, 40%);  
}  
  
/* HSLA Color */  
.hsla-color {  
  color: hsla(300, 100%, 30%, 0.8);  
}  
  
</style>  
</head>  
<body>  
  <h1 class="color-name">Aditya University</h1>  
  <p class="hex-color">  
    Department of Artificial Intelligence and Machine Learning  
  </p>  
  <p class="rgb-color">  
    AIML focuses on intelligent systems, data analysis, and automation.  
  </p>  
  <p class="rgba-color">  
    Artificial Intelligence enables machines to mimic human intelligence.  
  </p>  
  <p class="hsl-color">  
    Machine Learning allows systems to learn from data and improve over time.  
  </p>  
  <p class="hsla-color">  
    Aditya University AIML prepares students for future technologies.  
  </p>  
</body>
```

</html>

Aditya University

Department of Artificial Intelligence and Machine Learning

AIML focuses on intelligent systems, data analysis, and automation.

Artificial Intelligence enables machines to mimic human intelligence.

Machine Learning allows systems to learn from data and improve over time.

Aditya University AIML prepares students for future technologies.

d. Write a program, to explain the importance of CSS Box model using
i. Content ii. Border iii. Margin iv. Padding

In CSS, every HTML element is represented as a **box**.

The **CSS Box Model** defines how the size, spacing, and layout of elements are calculated on a web page.

Understanding the box model is essential for proper **web page design, layout control, and responsive styling**.

The CSS Box Model consists of the following parts:

1. **Content**
2. **Padding**
3. **Border**
4. **Margin**

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>CSS Box Model</title>
```

```
<style>
```

```
.box {
```

```
width: 300px;
```

```
/* Content */
```

```
background-color: lightyellow;
```

```
font-size: 18px;
```

```
text-align: center;

/* Padding */

padding: 20px;

/* Border */

border: 5px solid blue;

/* Margin */

margin: 30px;

}

</style>

</head>

<body>

  <h1>CSS Box Model Example</h1>

  <div class="box">

    Content Area

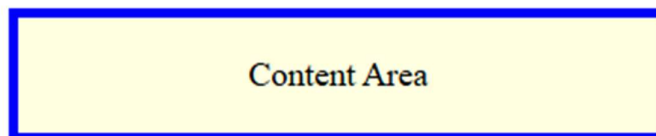
  </div>

</body>

</html>
```



CSS Box Model Example



3. Selector Forms & JavaScript – Internal and I/O

- a. Write a program to apply different types of selector forms:
 - i. Simple selector (element, id, class, group, universal)
 - ii. Combinator selector (descendant, child, adjacent sibling, general sibling)

JavaScript is a high-level, interpreted, and object-based scripting language used to make web pages **dynamic and interactive**.

While **HTML** defines the structure of a web page and **CSS** controls the appearance, **JavaScript adds behavior** such as responding to user actions, validating forms, manipulating content, and communicating with servers.

◆ Importance of JavaScript

- Enables **dynamic content updates**
- Handles **user events** (clicks, input, mouse actions)
- Performs **form validation**
- Creates **interactive web applications**
- Used in frontend, backend, mobile apps, and servers

◆ Role of CSS with JavaScript

JavaScript often works together with **CSS selectors** to select and manipulate HTML elements.

Understanding CSS selectors is essential for effective JavaScript DOM manipulation.

CSS selectors are patterns used to **select HTML elements** and apply styles to them. They help in targeting elements efficiently and precisely.

Simple Selectors

Selector Type	Description
Element	Selects HTML elements (p, h1)
ID	Selects unique element (#main-title)
Class	Selects multiple elements (.highlight)
Group	Selects multiple elements together
Universal	Selects all elements (*)

Combinator Selectors

Selector	Description
Descendant (A B)	Selects elements inside another
Child (A > B)	Selects direct children only
Adjacent Sibling (A + B)	Selects next sibling
General Sibling (A ~ B)	Selects all siblings

```
<<!DOCTYPE html>
<html>
<head>
  <title>CSS Selector Forms</title>
  <style>
    /* Page background */
    body {
      background-color: #f2f6ff;
    }
  </style>
</head>
</html>
```

```
/* ===== */
/* SIMPLE SELECTORS */
/* ===== */

/* Element Selector */
p {
  color: blue;
}

/* ID Selector */
#main-title {
  color: darkred;
  text-align: center;
}

/* Class Selector */
.highlight {
  background-color: lightyellow;
}

/* Group Selector */
h2, h3 {
  color: green;
}

/* Universal Selector */
* {
  font-family: Arial, sans-serif;
}

/* ===== */
/* COMBINATOR SELECTORS */
/* ===== */

/* Descendant Selector */
div p {
  font-style: italic;
}

/* Child Selector */
div > span {
  color: purple;
  font-weight: bold;
}

/* Adjacent Sibling Selector */
h3 + p {
  color: orange;
}
```

```

    /* General Sibling Selector */
    h3 ~ p {
        background-color: #e0e0e0;
    }
</style>
</head>
<body>

    <h1 id="main-title">CSS Selector Demonstration</h1>

    <h2>Simple Selectors</h2>

    <p>This paragraph uses Element Selector</p>
    <p class="highlight">This paragraph uses Class Selector</p>

    <h3>Group Selector Example</h3>

    <div>
        <p>Descendant Selector applied here</p>
        <span>Child Selector applied here</span>
    </div>

    <h2>Combinator Selectors</h2>

    <h3>Adjacent and General Sibling</h3>
    <p>This paragraph is adjacent sibling</p>
    <p>This paragraph is general sibling</p>
    <p>This paragraph is also general sibling</p>

</body>
</html>

```

CSS Selector Demonstration

Simple Selectors

This paragraph uses Element Selector

This paragraph uses Class Selector

Group Selector Example

Descendant Selector applied here

Child Selector applied here

Combinator Selectors

Adjacent and General Sibling

This paragraph is adjacent sibling

This paragraph is general sibling

This paragraph is also general sibling

b. Write a program to embed internal and external JavaScript in a web page.

Types of JavaScript Embedding

Internal JavaScript

- Written inside the HTML file
- Placed within the <script> tag
- Useful for small programs and testing

External JavaScript

- Written in a separate .js file
- Linked to HTML using <script src="">
- Improves code reusability and maintenance

js_example/

```
├── index.html
└── script.js
```

```
<html>
<head>
  <title>Internal and External JavaScript</title>

  <!-- Internal JavaScript alert() Displays a popup message-->
  <script>
    function internalMessage() {
      alert("This is Internal JavaScript");
    }
  </script>
</head>
<body>

  <h1>JavaScript Embedding Example</h1>

  <button onclick="internalMessage()">
    Internal JavaScript
  </button>

  <br><br>

  <button onclick="externalMessage()">
    External JavaScript
  </button>

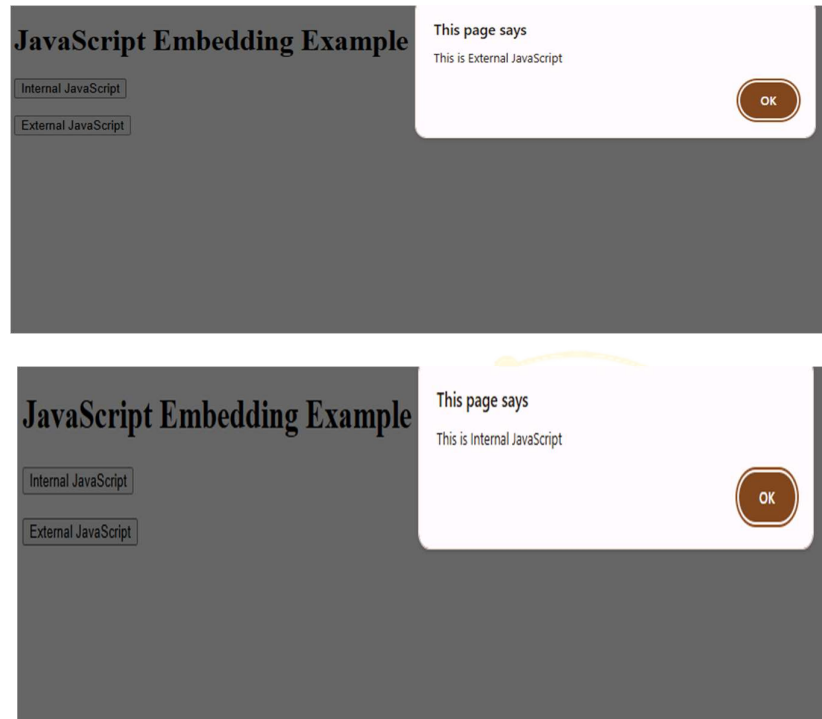
  <!-- External JavaScript -->
  <script src="script.js"></script>

</body>
</html>
```

Step 2: External JavaScript FileSave this file as **script.js**

(Important: extension must be .js)

```
function externalMessage() {
  alert("This is External JavaScript");
}
```



c. Write a program to explain the different ways for displaying output.

JavaScript provides multiple ways to display output to the user.

These methods are used to show messages, results of calculations, or dynamic content on a web page.

Different output methods are used depending on:

- 🚦 User interaction
- 🚦 Debugging
- 🚦 Web page content display

Different Ways of Displaying Output in JavaScript

1. Using **alert()**
2. Using **document.write()**
3. Using **innerHTML**
4. Using **console.log()**

```
<!DOCTYPE html>
<html>
<head>
  <title>JavaScript Output Methods</title>
```

```
<script>
  function showAlert() {
    alert("This output is displayed using alert()");
  }

  function writeDocument() {
    document.write("This output is displayed using document.write()");
  }

  function showInnerHTML() {
    document.getElementById("output").innerHTML =
      "This output is displayed using innerHTML";
  }

  function showConsole() {
    console.log("This output is displayed using console.log()");
  }
</script>
</head>
<body>

  <h1>Different Ways of Displaying Output in JavaScript</h1>

  <button onclick="showAlert()">Alert Output</button>
  <br><br>

  <button onclick="writeDocument()">Document Write Output</button>
  <br><br>

  <button onclick="showInnerHTML()">InnerHTML Output</button>
  <p id="output"></p>

  <button onclick="showConsole()">Console Output</button>

</body>
</html>
```

Different Ways of Displaying Output in JavaScript

Alert Output

Document Write Output

InnerHTML Output

Console Output

4. JavaScript Pre-defined and User-defined Objects

(a) Program using document, window, Math & String object properties and methods

Aim

To demonstrate the use of JavaScript **pre-defined objects** such as **document**, **window**, **Math**, and **String** along with their properties and methods.

Explanation

- **window.prompt()** → Accepts input from user
- **String methods** → toUpperCase(), length
- **Math methods** → sqrt(), pow()
- **document.getElementById()** → Displays output on web page

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>JavaScript Objects Demo</title>
```

```
</head>
```

```
<body>
```

```
  <h3>JavaScript Pre-defined Objects</h3>
```

```
  <p id="result"></p>
```

```
<script>
```

```
  // Window Object
```

```
  window.alert("Welcome to JavaScript Objects Program");
```

```
  let name = window.prompt("Enter your name:");
```

```
  // String Object
```

```
  let upperName = name.toUpperCase();
```

```

let lengthOfName = name.length;

// Math Object
let number = 25;
let sqrtValue = Math.sqrt(number);
let powerValue = Math.pow(5, 2);

// Document Object
document.getElementById("result").innerHTML =
  "Name: " + name + "<br>" +
  "Uppercase Name: " + upperName + "<br>" +
  "Length of Name: " + lengthOfName + "<br>" +
  "Square Root of 25: " + sqrtValue + "<br>" +
  "5 Power 2: " + powerValue;
</script>

</body>
</html>

```

JavaScript Pre-defined Objects

Name: kalyan
 Uppercase Name: KALYAN
 Length of Name: 6
 Square Root of 25: 5
 5 Power 2: 25

(b) Program to find the LARGER NUMBER among three integers

Aim

To accept three integers from the user and display the largest number using an information message dialog.

If all numbers are equal, display “EQUAL NUMBER”.

Explanation

- **prompt()** → Accepts user input
- **parseInt()** → Converts string to integer
- **if-else if-else** → Compares three numbers
- **alert()** → Displays information dialog

```

<!DOCTYPE html>
<html>
<head>
  <title>Larger Number</title>
</head>
<body>

<script>
  let a = parseInt(prompt("Enter first number:"));

```



```

let b = parseInt(prompt("Enter second number:"));
let c = parseInt(prompt("Enter third number:"));

if (a > b && a > c) {
  document.write(a + " LARGER NUMBER");
  alert(a + " LARGER NUMBER");
}
else if (b > a && b > c) {
  document.write(b + " LARGER NUMBER");
  alert(b + " LARGER NUMBER");
}
else if (c > a && c > b) {
  document.write(c + " LARGER NUMBER");
  alert(c + " LARGER NUMBER");
}
else {
  document.write("EQUAL NUMBER");
  alert("EQUAL NUMBER");
}
</script>

</body>
</html>

```

6 LARGER NUMBER

(c) Program to print 1 to 10 using for, while, and do-while loops

Aim

To print numbers from **1 to 10** using different loop statements in JavaScript.

Explanation

Loop Type	Description
for loop	Executes fixed number of times
while loop	Executes while condition is true
do-while loop	Executes at least once

```

<!DOCTYPE html>

<html>

<head>

  <title>Loops in JavaScript</title>

</head>

<body>

```

<h3>For Loop</h3>

<script>

```
for (let i = 1; i <= 10; i++) {  
  document.write(i + " ");  
}
```

</script>

<h3>While Loop</h3>

<script>

```
let j = 1;  
while (j <= 10) {  
  document.write(j + " ");  
  j++;  
}
```

</script>

<h3>Do-While Loop</h3>

<script>

```
let k = 1;  
do {  
  document.write(k + " ");  
  k++;  
} while (k <= 10);
```

</script>

</body>

</html>



ADITYA
UNIVERSITY

For Loop

1 2 3 4 5 6 7 8 9 10

While Loop

1 2 3 4 5 6 7 8 9 10

Do-While Loop

1 2 3 4 5 6 7 8 9 10

5. JavaScript Functions and Events & Node.js

(a) Program using functions to display

- i) Factorial of a number
- ii) Fibonacci series up to that number

Aim

To design JavaScript functions to compute the factorial of a number and generate the Fibonacci series up to a given number.

Explanation

- **factorial()** → Calculates factorial using for loop
- **fibonacci()** → Generates Fibonacci series
- **onclick event** → Calls function on button click

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>Factorial and Fibonacci</title>
```

```
</head>
```

```
<body>
```

```
  <h3>Factorial and Fibonacci Series</h3>
```

Enter a number:

```
<input type="number" id="num">
```

```
<button onclick="calculate()">Calculate</button>
```

```
<p id="result"></p>
```

```
<script>
```

```
  // Function to calculate factorial
```

```
  function factorial(n) {
```

```
    let fact = 1;
```

```
    for (let i = 1; i <= n; i++) {
```

```

        fact *= i;
    }
    return fact;
}

// Function to generate Fibonacci series
function fibonacci(n) {
    let a = 0, b = 1, temp;
    let series = "0 1 ";

    for (let i = 2; i < n; i++) {
        temp = a + b;
        series += temp + " ";
        a = b;
        b = temp;
    }
    return series;
}

function calculate() {
    let n = parseInt(document.getElementById("num").value);

    let factResult = factorial(n);
    let fibResult = fibonacci(n);

    document.getElementById("result").innerHTML =
        "Factorial of " + n + " is: " + factResult + "<br>" +
        "Fibonacci Series: " + fibResult;
}
</script>

</body>
</html>

```

Factorial and Fibonacci Series

Enter a number:

Factorial of 8 is: 40320

Fibonacci Series: 0 1 1 2 3 5 8 13

(b) Program to show workflow of JavaScript execution using Node.js Web Server

Aim

To demonstrate the execution of JavaScript code by creating a **web server using Node.js**.

```
const http = require('http');

// Create Web Server

const server = http.createServer((req, res) => {

  res.writeHead(200, { 'Content-Type': 'text/html' });

  res.write('<h2>Node.js Web Server</h2>');

  res.write('<p>JavaScript code executed on the server</p>');

  res.end();

});

// Server listens on port 3000

server.listen(3000, () => {

  console.log("Server running at http://localhost:3000");

});
```

Steps to Execute

1. Install Node.js
2. Save file as server.js
3. Open terminal
4. Run command:
5. node server.js

Open browser → <http://localhost:3000>

Explanation

- require('http') → Imports HTTP module
- createServer() → Creates web server
- listen() → Starts server on port 3000

Introduction to Node.js

Node.js is a **runtime environment** that allows you to run **JavaScript outside the browser**, mainly on the **server side**.

It is built on **Chrome's V8 JavaScript engine** and is used to create **fast, scalable, and efficient web applications**.

In simple words 👉

Node.js lets us use JavaScript to build backend/server applications.

JavaScript Everywhere

- Same language (JavaScript) for both frontend and backend.

Fast Execution

- Uses Google Chrome's V8 engine → very fast.

Asynchronous & Non-Blocking

- Handles multiple requests at the same time without waiting.

Single-Threaded

- Uses one main thread but handles many users efficiently.

Event-Driven

- Works based on events (request, response, callback).

NPM (Node Package Manager)

- Large collection of free libraries and tools.

Used for Backend Development

- APIs, web servers, real-time apps, etc.

c. Write a program to transfer data over HTTP protocol using HTTP module.

```
const http = require('http');
```

```
const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.write('Hello Client! Data transferred using HTTP protocol.');
```

```
  res.end();
});
```

```
server.listen(3000, () => {
  console.log('Server running at http://localhost:3000/');
});
```

CLIENT PROGRAM

```
const http = require('http');

const options = {
  hostname: 'localhost',
  port: 3000,
  path: '/',
  method: 'GET'
};

const req = http.request(options, (res) => {
  let data = "";

  res.on('data', (chunk) => {
    data += chunk;
  });

  res.on('end', () => {
    console.log('Received from server:', data);
  });
});

req.on('error', (error) => {
  console.error(error);
});

req.end();
```

d. Create a text file **src.txt** and add the following content to it:
(HTML, CSS, JavaScript, TypeScript, MongoDB, Express.js, React.js, Node.js)

File System Module (fs) in Node.js

The **File System (fs) module** in Node.js is used to **work with files and directories** on the computer.

It allows us to **create, read, write, update, delete, and manage files**.

👉 In simple words:

The fs module helps Node.js interact with files on the system.

```
const fs = require('fs');
```

Commonly Used fs Module Functions **Create / Write File**

```
fs.writeFile('file.txt', 'Hello', callback);
```

 Append Data

```
fs.appendFile('file.txt', ' World', callback);
```

 Read File

```
fs.readFile('file.txt', 'utf8', callback);
```

 Delete File

```
fs.unlink('file.txt', callback);
```

 Create Folder

```
fs.mkdir('myFolder', callback);
```

 Read Folder

```
fs.readdir('myFolder', callback);
```

```
const fs = require('fs');
const path = require('path');
```

```
// Create file in the same folder as this JS file
const filePath = path.join(__dirname, 'src.txt');
```

```
const content = "HTML, CSS, JavaScript, TypeScript, MongoDB, Express.js, React.js, Node.js";
```

```
fs.writeFile(filePath, content, (err) => {
  if (err) {
    console.error("Error:", err);
  } else {
    console.log("src.txt file created successfully");
  }
});
```

6. TypeScript

a. Write a program to understand simple and special types.

```
// Simple Types
```

```
let studentId: number = 101;
```

```
let studentName: string = "Rahul";
```

```
let isPresent: boolean = true;
```

```
// Special Types
```

```
let valueAny: any = "Hello";
```

```
valueAny = 123; // allowed
```

```
let valueUnknown: unknown = "TypeScript";
```

```
// void type (used in functions)
```

```
function displayMessage(): void {
  console.log("Welcome to TypeScript");
}
```



```
// null and undefined
let emptyValue: null = null;
let notAssigned: undefined = undefined;
```

```
// Output
console.log(studentId);
console.log(studentName);
console.log(isPresent);
displayMessage();
```

b. Write a program to understand function parameter and return types.

// Simple Types

```
let studentId: number = 101;
```

```
let studentName: string = "Manikanta";
```

```
let isPresent: boolean = true;
```

```
let marks: number[] = [85, 90, 88]; // Array
```

```
let studentInfo: [number, string] = [101, "Kalyan"]; // Tuple
```

// Special Types

```
let valueAny: any = "Hello";
```

```
valueAny = 123; // Allowed
```

```
let valueUnknown: unknown = "TypeScript";
```

```
// valueUnknown.toUpperCase(); // Error without type check
```

```
let resultVoid: void;
```

```
resultVoid = undefined;
```

```
let emptyValue: null = null;
```

```
let notAssigned: undefined = undefined;
```

```
// Never type example
```

```
function throwError(message: string): never {
  throw new Error(message);
}
```

```
// Output
```

```
console.log("ID:", studentId);
console.log("Name:", studentName);
console.log("Present:", isPresent);
console.log("Marks:", marks);
console.log("Student Info:", studentInfo);
```

7. ExpressJS – Routing, HTTP Methods, Templating

a. Write a program to define a route, handling routes, route parameters, query parameters and URL building.

```
const express = require('express');
const app = express();
const PORT = 3000;

// Basic Route
app.get('/', (req, res) => {
  res.send('Welcome to ExpressJS Routing');
});

// Route Parameter
app.get('/student/:id', (req, res) => {
  res.send(`Student ID: ${req.params.id}`);
});

// Multiple Route Parameters
app.get('/course/:name/:duration', (req, res) => {
  const { name, duration } = req.params;
  res.send(`Course: ${name}, Duration: ${duration}`);
});

// Query Parameters
app.get('/search', (req, res) => {
  const { subject, level } = req.query;
  res.send(`Subject: ${subject}, Level: ${level}`);
});

// URL Building
app.get('/build-url', (req, res) => {
```

```
const url = `/student/101?role=monitor`;
res.send(`Generated URL: ${url}`);
});
```

```
app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

b. Write a program to accept data, retrieve data and delete a specified resource using HTTP methods.

```
const express = require('express');
const app = express();
const PORT = 3000;
```

```
app.use(express.json()); // to read JSON data
```

```
let items = []; // temporary storage
```

```
// POST → Add data
```

```
app.post('/items', (req, res) => {
  const item = req.body;
  items.push(item);
  res.send('Item added successfully');
});
```

```
// GET → Retrieve all data
```

```
app.get('/items', (req, res) => {
  res.json(items);
});
```

```
// DELETE → Delete specific item by index
```

```
app.delete('/items/:index', (req, res) => {
  const index = req.params.index;
  items.splice(index, 1);
  res.send('Item deleted successfully');
});
```

```
app.listen(PORT, () => {
```

```
  console.log(`Server running on port ${PORT}`);
});
```

Node app.js

Open in browser : <http://localhost:3000/items>

Invoke-RestMethod `

-Uri http://localhost:3000/items `

-Method POST `

-ContentType "application/json" `

-Body '{"name":"Book"}'

c. Write a program using templating engine.

npm install ejs

```
const express = require('express');

const app = express();

const PORT = 3000;

app.set('view engine', 'ejs');

// Route to render template
app.get('/', (req, res) => {

  const user = {

    name: "Manikanta",

    course: "ExpressJS"

  };

  res.render('home', { user });

});

app.listen(PORT, () => {

  console.log(`Server running on port ${PORT}`);

});
```

home.ejs

```
<!DOCTYPE html>

<html>

<head>

  <title>Home Page</title>

  <style>

    body {

      background-color: lightblue; /* change color here */

    }

  </style>
```

```
</head>

<body>

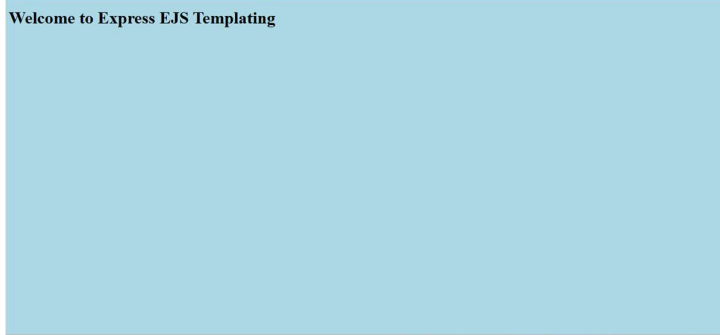
  <h1>Welcome to Express EJS Templating</h1>

</body>

</html>

node app1.js
```

Open in browser : <http://localhost:3000>



8. ExpressJS – Cookies, Sessions, Authentication, Database

- Write a program for session management using cookies and sessions.
- Write a program for user authentication.
- Write a program to connect MongoDB database using Mongoose and perform CRUD operations.

Write a program for session management using cookies and sessions.

1. Cookies store small user data in the browser (client side).
2. Sessions store user information on the server.
3. A session ID is stored in cookies to identify the user.

To maintain and track user information across multiple requests by storing session data on the server and using cookies to identify the user.

npm install express-session

npm install cookie-parser

```
const express = require('express');
const session = require('express-session');
const cookieParser = require('cookie-parser');
const app = express();
const PORT = 3000;
app.use(cookieParser());
// session middleware
app.use(session({
  secret: "mysecretkey",
  resave: false,
  saveUninitialized: true
}));
// home route
app.get('/', (req, res) => {
  if (req.session.views) {
    req.session.views++;
    res.send(`Visited ${req.session.views} times`);
  } else {
    req.session.views = 1;
    res.send("Welcome! First visit");
  }
});

// destroy session
app.get('/logout', (req, res) => {
  req.session.destroy();
  res.send("Session ended");
});
```

```
});
```

```
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Write a program for user authentication.

1. User authentication verifies username and password before allowing access to the system.
2. Sessions are used to store login information after successful authentication.
3. Protected routes can only be accessed when the user is logged in.
4. If login credentials are invalid, access is denied and an error message is shown.
5. Logout destroys the session and removes user access to protected pages.

Shell command

npm install express express-session body-parser

```
const express = require('express');
const session = require('express-session');
const bodyParser = require('body-parser');
const app = express();
const PORT = 3000;

// middleware
app.use(bodyParser.urlencoded({ extended: true }));
app.use(session({
  secret: "mysecretkey",
  resave: false,
  saveUninitialized: false
}));
```

```
// login page
```

```

app.get('/', (req, res) => {
  res.send(`
    <h2>Login Page</h2>
    <form method="POST" action="/login">
      <input type="text" name="username" placeholder="Username" required /><br><br>
      <input type="password" name="password" placeholder="Password" required /><br><br>
      <button type="submit">Login</button>
    </form>
  `);
});

// login authentication
app.post('/login', (req, res) => {
  const { username, password } = req.body;

  // sample credentials
  if (username === "admin" && password === "1234") {
    req.session.user = username;
    res.send("Login successful <br><a href='/dashboard'>Go to Dashboard</a>");
  } else {
    res.send("Invalid username or password");
  }
});

// protected page
app.get('/dashboard', (req, res) => {
  if (req.session.user) {
    res.send(`Welcome ${req.session.user} <br><a href="/logout">Logout</a>`);
  }
});

```



```

    } else {
      res.send("Please login first");
    }
  });

// logout
app.get('/logout', (req, res) => {
  req.session.destroy();
  res.send("Logged out successfully");
});

app.listen(PORT, () => {
  console.log("Server running on http://localhost:3000");
});

```

Username: admin

Password: 1234

Write a program to connect MongoDB database using Mongoose and perform CRUD operations.

Create → Add new data

Used to insert new records into the database.

Example: Add a new student

Name: Ravi

Age: 21

Read → View data

Used to retrieve or display data from the database.

Example: Show all students list

In simple words: **see stored information**

Update → Modify data

Used to change existing data in the database.

Example: Change Ravi's age from 21 to 22

In simple words: **edit stored information**

Delete → Remove data

Used to delete records from the database.

Example: Remove Ravi from student list

In simple words: **remove stored information**

Install packages

npm install express mongoose body-parser

Start MongoDB server

9. ExpressJS – Templating, Form Data, Cookies, Sessions

a. Write a program using templating engine.

A **Templating Engine** allows you to generate dynamic HTML pages by combining:

- Static HTML
- Dynamic data (variables, loops, conditions)

Common templating engines used with Express:

- **EJS (Embedded JavaScript)**
- Pug
- Handlebars

Here we use **EJS**

Why Templating Engine?

- To pass data from server to HTML page
- To display dynamic content
- To reuse layouts

```
const express = require('express');
```

```
const app = express();
```

```
app.set('view engine', 'ejs');
```

```
app.get('/', (req, res) => {
  res.render('index', {
    name: "Manikanta",
    course: "MERN Stack"
  });
});
```

```
app.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});
```

```
App9.ejs
<!DOCTYPE html>
<html>
<head>
  <title>Templating Example</title>
</head>
<body>
  <h1>Welcome <%= name %></h1>
  <h2>Course: <%= course %></h2>
</body>
</html>
```

- b. Write a program to work with form data.
- c. Write a program for session management using cookies and sessions.

ReactJS – Render HTML, JSX, Props and States

npm create vite@latest react-app

Select Framework:

React

Select Variant:

JavaScript

npm run dev

Introduction to ReactJS

React is a JavaScript library used to build user interfaces (UI), especially for single-page applications. It allows developers to create reusable components and manage dynamic data efficiently.

Key Features of React:

- Component-based architecture
- Virtual DOM for fast updates
- One-way data binding
- Reusable UI components

- a. Write a program to render HTML to a web page.

Render HTML in React

Theory

Rendering HTML means displaying content on a web page using React.

In traditional HTML, we directly write HTML inside a file.

In React, we use JavaScript to render HTML using:

- ReactDOM.createRoot()
- .render() method

React renders content inside a root div (usually <div id="root"></div> in index.html).

Important Points:

- React uses a root element to display content.
- The render() method displays elements on the web page.
- React converts JSX/HTML into JavaScript objects internally.

- b. Write a program for writing markup with JSX.

c. Write a program to work with props and states.

10(a) Render HTML Program

```
function App() {
  return (
    <div>
      <h1>Welcome to React</h1>
      <p>This HTML is rendered using React.</p>
    </div>
  );
}
```

export default App;

Save → Browser auto refreshes

10(b) JSX Program

Replace again with:

```
function App() {

  const name = "Manikanta";
  const marks = 85;

  return (
    <div>
      <h1>Hello {name}</h1>
      <h2>Your Marks: {marks}</h2>
      <h3>{marks > 50 ? "Pass" : "Fail"}</h3>
    </div>
  );
}
```

export default App;

10(c) Props and State Program

Replace with:

```
import { useState } from "react";

function Student(props) {
  return (
    <div>
      <h2>Name: {props.name}</h2>
      <h3>Marks: {props.marks}</h3>
    </div>
  );
}
```

```

}

function App() {

  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>Props and State Example</h1>

      <Student name="Manikanta" marks={90} />

      <h2>Counter: {count}</h2>

      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </div>
  );
}

```

export default App;

Click button → Counter increases

npm run dev 

(a) Program for Conditional Rendering

Conditional rendering means displaying different UI based on a condition.

In React, we use:

- if statement
- Ternary operator ? :
- Logical AND &&

Program (Using Ternary Operator)

Open src/App.jsx and write:

```
import { useState } from "react";
```

```

function App() {

  const [isLoggedIn, setIsLoggedIn] = useState(false);

  return (
    <div>

```

```

<h1>Conditional Rendering Example</h1>

{isLoggedIn ? (
  <h2>Welcome Manikanta!</h2>
) : (
  <h2>Please Login</h2>
)}

<button onClick={() => setIsLoggedIn(!isLoggedIn)}>
  Toggle Login
</button>
</div>
);
}

```

export default App;

Output:

Click button → Text changes.

(b) Program for Rendering Lists

React uses .map() function to render lists.

Each item must have a **unique key**.

Program

Replace App.jsx:

```

function App() {

  const students = ["Manikanta", "Ravi", "Kiran", "Suresh"];

  return (
    <div>
      <h1>Rendering List Example</h1>

      <ul>
        {students.map((student, index) => (
          <li key={index}>{student}</li>
        ))}
      </ul>

    </div>
  );
}

```

export default App;

Output:

List of names displayed.

(c) Routing Using React Router

Now we create multiple pages.

Step 1: Install React Router

Inside project folder:

```
npm install react-router-dom
```

Step 2: Create Pages

Inside src, create 3 files:

 Home.jsx

```
function Home() {  
  return <h2>Home Page</h2>;  
}
```

```
export default Home;
```

 About.jsx

```
function About() {  
  return <h2>About Page</h2>;  
}
```

```
export default About;
```

Contact.jsx

```
function Contact() {  
  return <h2>Contact Page</h2>;  
}
```

```
export default Contact;
```

Step 3: Modify App.jsx

Replace App.jsx with:

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";  
import Home from "./Home";  
import About from "./About";  
import Contact from "./Contact";
```

```
function App() {  
  return (  

```

```

<BrowserRouter>
  <div>
    <h1>React Router Example</h1>

    <nav>
      <Link to="/">Home</Link> |
      <Link to="/about"> About</Link> |
      <Link to="/contact"> Contact</Link>
    </nav>

    <Routes>
      <Route path="/" element={<Home />} />
      <Route path="/about" element={<About />} />
      <Route path="/contact" element={<Contact />} />
    </Routes>

  </div>
</BrowserRouter>
);
}

```

export default App;

(Routing)

React Router is used to navigate between pages without refreshing the browser.

Main components:

- BrowserRouter
- Routes
- Route
- Link

React Router works with React library developed by Facebook.

Viva Points

Conditional Rendering:

Displays UI based on condition.

Rendering Lists:

Uses .map() function and unique key.

Routing:

Used to navigate between components using React Router.

a) Program to Understand the Importance of Using Hooks

Hooks allow functional components to use **state and lifecycle features** without writing class components.

The most commonly used hook is **useState()**, which helps manage component state.

Program (useState Hook Example)

```
import { useState } from "react";

function App() {

  const [count, setCount] = useState(0);

  return (
    <div>
      <h1>React Hooks Example</h1>

      <h2>Counter: {count}</h2>

      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>

      <button onClick={() => setCount(count - 1)}>
        Decrease
      </button>

    </div>
  );
}

export default App;
```

Explanation

- useState() creates state variable count
- setCount() updates the state
- React re-renders UI automatically

b) Program for Sharing Data Between Components

Data sharing between components is done using **Props**.
The **parent component passes data to child components**.

Program

```

function Child(props) {
  return (
    <div>
      <h2>Student Name: {props.name}</h2>
      <h3>Course: {props.course}</h3>
    </div>
  );
}

function App() {
  return (
    <div>
      <h1>Sharing Data Between Components</h1>

      <Child name="Manikanta" course="ReactJS" />

    </div>
  );
}

export default App;

```

Explanation

- App is parent component
- Child receives data using props
- Data flows **Parent** → **Child**

c) To-Do List Application

A To-Do list application allows users to **add tasks dynamically**. React state is used to store and update the list.

Program

```

import { useState } from "react";

function App() {

  const [task, setTask] = useState("");
  const [tasks, setTasks] = useState([]);

  const addTask = () => {
    setTasks([...tasks, task]);
    setTask("");
  };

  return (

```

```

<div>

  <h1>To-Do List</h1>

  <input
    type="text"
    value={task}
    onChange={(e) => setTask(e.target.value)}
    placeholder="Enter task"
  />

  <button onClick={addTask}>Add</button>

  <ol>
    {tasks.map((t, index) => (
      <li key={index}>{t}</li>
    ))}
  </ol>

</div>
);
}

```

export default App;

Features

- Add new task
- Display task list
- Uses useState hook

d) Install MongoDB and Configure Atlas

MongoDB is a **NoSQL database** used to store data in JSON-like documents.

MongoDB Atlas is a **cloud version of MongoDB**.

Installation Steps

Step 1: Download MongoDB

Go to official website:

<https://www.mongodb.com/try/download/community>

Install MongoDB Community Edition.

Step 2: Install MongoDB Compass

MongoDB Compass is GUI tool to manage databases.

Step 3: Create Atlas Account

Go to:

<https://www.mongodb.com/atlas>

Create free account.

Step 4: Create Cluster

1. Click **Build Cluster**
2. Select **Free Tier**
3. Choose cloud provider
4. Create cluster

Step 5: Create Database User

Go to:

Database Access → Add New User

Set username and password.

Step 6: Allow Network Access

Go to:

Network Access → Add IP Address

Add:

0.0.0.0/0

Step 7: Connect to Database

Click:

Connect → Connect your application

You will get connection string like:

mongodb+srv://username:password@cluster.mongodb.net/myDatabase

